

Project

[Start Assignment](#)

- Due Thursday by 11:59pm
- Points 100
- Submitting a text entry box, a website url, or a file upload
- Available Jun 22 at 12am - Jul 2 at 11:59pm

Course Project — End-to-End Machine Learning, Built with AI

Your capstone. Take a real dataset of your choice from raw data all the way to a live, deployed prediction app — using everything from the course: the full ML pipeline (Parts 1–3) and the AI-assisted workflow (Claude Code + MCP servers).

This is an **individual** project. Everything you need is in this one assignment.

What this project is

You will run a **complete, honest machine-learning study** on a dataset **you choose**, answering **two questions** about it — one **classification** question and one **regression** question — and then **ship your model as a live web app**. You work the way professionals now do: **AI-assisted**, with Claude Code and MCP servers, while *verifying everything yourself*.

Think of the workshop's worked example (`ml-analysis.qmd`) as your template — applied to your own data, in more depth.

Your dataset → EDA & cleaning → Feature engineering + selection → Leakage-safe split → Model ladder: baseline → XGBoost → Benchmark leaderboard + honest evaluation → Deployed web app (Streamlit or React) + Report · slides · summary

Phase 1 — Proposal & dataset approval (required before you start)

Because you bring your own data, you must get it **approved first** so nobody sinks a week into an unworkable dataset.

Submit a short proposal ($\frac{1}{2}$ page) containing:

1. **The dataset** — source/link, roughly how many rows and columns, and that it is public/shareable (no private or personal data).
2. **Your classification question** — and which column is (or will be derived as) the **categorical target**.
3. **Your regression question** — and which column is (or will be derived as) the **numeric target**.

4. Why it is interesting — one or two sentences on the real-world question.

Your dataset must:

- Have **at least ~500 rows** and a **mix of numeric and (ideally) categorical features**.
- Support **both** a classification target **and** a regression target — either naturally, or **derived** from the data (like the worked example, where "will the customer return?" was classification and "how valuable will they become?" was regression).
- **Not** be one of the datasets used in lectures (no Wine Quality, no California Housing) and not a trivial toy dataset.

✅ Wait for approval before moving on. Good sources: UCI ML Repository, Kaggle, data.gov, Our World in Data, OpenML.

Phase 2 — The analysis (the heart of the project)

Work through the **whole pipeline from the course**. Each step maps to material you have already seen.

A. Understand & clean (Part 1)

- Load and inspect the data; document its shape, types, and quirks.
- **EDA**: distributions, target analysis, a correlation study, and feature–target relationships, with clear visualizations.
- **Clean it**: handle missing values (justify your strategy), detect and treat outliers, and scale features where the model needs it (Standard / MinMax / Robust).

B. Engineer & select features (Part 2) — *leakage-safe*

- Create **meaningful engineered features** (ratios, interactions, domain logic, binning, polynomial where sensible).
- Apply at least one **feature-selection** method (correlation filter, RFE, variance threshold, or Lasso).
- 🚫 **No data leakage**. Fit scalers/encoders/selectors **inside** the training split only (use a `Pipeline` / `ColumnTransformer`). If you derive features and targets from history, derive **features from the past and targets from the future**, like the worked example.

C. Split & validate properly (Part 2)

- Use a **train / validation / test** split or **cross-validation**. Use **stratified** splits for classification.
- Briefly show you understand **overfitting vs underfitting** for at least one model (a learning or validation curve, or a train-vs-test gap).

D. Build a model ladder for BOTH tasks (Parts 2 & 3)

For **each** task (classification *and* regression), train a ladder from simple to advanced and compare honestly:

1. A **baseline** — `DummyClassifier` / `DummyRegressor` (you must beat this).
2. A **linear model** — Logistic Regression / Linear or Ridge/Lasso Regression.

3. A **tree** — Decision Tree (and note where it overfits).

4. **XGBoost** — with **cross-validation** and some **hyperparameter tuning**.

E. Benchmark & evaluate honestly (Part 3)

- Produce a **leaderboard table** comparing all models on the **test set**, using the **right metrics**:
 - Classification: accuracy, precision/recall/F1, ROC-AUC, confusion matrix.
 - Regression: R^2 , MAE, RMSE; a predicted-vs-actual plot.
- **Honest evaluation philosophy (graded heavily)**: always report **test** performance (not just train), keep the **baseline** in the table, **call out overfitting**, and **do not oversell**. If the model is only slightly better than the baseline, say so.
- **Interpret for the business**: what do the results mean for the real-world question? Which features matter? What would you *not* trust this model to do?

Deliverables (all five are required)

#	Deliverable	What it is
D1	Reproducible analysis report	A Quarto document (<code>.qmd</code> → HTML) with the full Phase-2 analysis: code, output, charts, prose. Must run end-to-end from a clean checkout.
D2	Deployed web app	A live app — Streamlit (Python) or React (JavaScript) — using your work: a predictor (user enters feature values → model returns a prediction) and/or an interactive findings dashboard. React is the higher-value option (scores higher on subjective marks). See "Deploying & sharing your app" below.
D3	AI-workflow reflection	½–1 page: which MCP servers / Claude Code features you used, how you verified the AI's output, and a rough sense of cost/effort .
D4	Presentation	A short slide deck (Quarto reveal.js) — problem → approach → findings — for a non-technical audience.
D5	Executive summary	One page , plain language: the question, what you found, what you would recommend. No jargon.

Everything lives in **one public GitHub repository** (pushed with the GitHub MCP), with a `README.md` that links the live app and explains how to run the report.

Deploying & sharing your app (D2)

You may build in **either** stack, but **React is the higher-value choice** — it scores higher on the subjective marks: it is a real web frontend you build and wire up yourself, not an auto-generated one. **Streamlit** is the simpler, safe option that still fully meets the requirement — pick it if you would rather spend your time on the modelling.

Option 1 — Streamlit (Python, simplest)

Your model runs *inside* the app, so there is no separate backend.

1. Put `app.py` + `requirements.txt` in your repo (push with the GitHub MCP).
2. **share.streamlit.io** → **Create app** → pick your repo, branch `main`, file `app.py` → **Deploy**.
3.  You get a public `https://...streamlit.app` link that loads with no errors. (Same flow as the home assignment.)

Option 2 — React (JavaScript; e.g. Vite + React)

A React app is a **frontend only**, so first decide how it gets your model's results:

- **2a — Static results (simplest):** export your model's outputs (predictions, leaderboard numbers, chart data) to a **JSON/CSV** file, commit it, and have React read it. No backend needed.
- **2b — Live predictions:** serve your trained model behind a small **FastAPI** endpoint and have React call it with `fetch`. More work, more impressive.

How to share a React app (free):

1. Build locally to check it works: `npm run build` (Vite outputs a `dist/` folder; preview with `npm run preview`).
2. Push the project to a **public GitHub repo** (GitHub MCP).
3. Connect that repo to **one** host — each gives a public URL and auto-redeploys on every push:

Host	How	Notes
Vercel (recommended)	vercel.com → Add New → Project → import your repo	Auto-detects Vite (build <code>npm run build</code> , output <code>dist</code>). Basically zero-config. → <code>*.vercel.app</code>
Netlify	netlify.com → Add new site → Import an existing project	Set Build command <code>npm run build</code> , Publish directory <code>dist</code> . → <code>*.netlify.app</code>
GitHub Pages	repo → Settings → Pages	Free but fiddlier: set Vite's <code>base</code> to <code>"/your-repo-name/"</code> and add a build-and-deploy GitHub Action.

 You get a public link that opens for anyone — test it in a private/incognito window.

 **If you serve live predictions (2b):** host the FastAPI part free on **Render**, **Railway**, or **Hugging Face Spaces**; enable **CORS** so your React site can call it; put the API's public URL in your React app. Keep any API keys in the host's **secret settings** — never in React code, which is fully visible in the browser.

 A **working React app earns the most**, but a clean **Streamlit** app that loads still beats an ambitious React app that crashes. Whatever you choose, **ship something that works**.

Technical & reproducibility requirements

- **Set a random seed** everywhere (`random_state`) so results are reproducible.
- Include a `requirements.txt`; the report and app must run from a clean install.
- **No leakage** — preprocessing fitted on training data only (use scikit-learn `Pipeline`).

- **AI is encouraged, understanding is assessed.** Use Claude Code/MCPs freely, but you must **understand and be able to explain every part** of your submission. You may be asked to walk through any section. Undisclosed copy-paste you cannot explain is treated as not your work.

Submission & self-check

Submit: your **GitHub repo URL**, your **live app URL**, and the **report, slides, and summary** (in the repo or as attachments — per the course platform).

Before submitting:

- ☐ Dataset was **approved** and supports both tasks.
- ☐ Report **runs end-to-end** from a clean checkout; seed is set.
- ☐ **No leakage** — preprocessing is inside the pipeline/training split.
- ☐ Both tasks have a **full model ladder incl. XGBoost** and a **leaderboard** on the **test set**.
- ☐ A **baseline** (Dummy) is in both leaderboards.
- ☐ Live app (Streamlit **or** React) **opens and works** for someone else.
- ☐ All five deliverables present; README links the app.
- ☐ AI-workflow reflection included.

Grading

☒ Objective

Area

Approved, suitable dataset (supports both tasks)

EDA + data cleaning (Part 1)

Feature engineering & selection, **leakage-free** (Part 2)

Proper validation — split/CV, stratified, no leakage

Classification ladder → XGBoost, right metrics, CV/tuning

Regression ladder → XGBoost, right metrics, CV/tuning

Benchmark leaderboard + **honest evaluation** (baseline kept, overfitting addressed, test reporting)

Reproducible public repo (seed, `requirements.txt`, runs end-to-end)

Deployed web app that loads & works — Streamlit *or* React (the stack choice is rewarded in the subjective score)

Presentation (problem → approach → findings)

Executive summary (1 page, plain language)

AI-workflow reflection (tools/MCPs, verification, cost)

Subjective (instructor's judgment)

- **Depth & rigor** — went beyond the minimum; thoughtful modeling choices; genuine insight from the data. **This is also where your app's technical ambition is rewarded:** a self-built,

deployed **React** app (especially with a live prediction API) demonstrates more than an auto-generated Streamlit app; a basic Streamlit app is perfectly acceptable but sits at the lower end of this axis.

- **Communication & polish** — report, slides, app, and summary are clear and professional; a non-technical reader could follow the story.
 - **Honesty & critical thinking** — avoided and discussed leakage; reported limitations; did not oversell; sensible real-world interpretation.
-

Tips

- **Beat the baseline first.** A model that cannot beat `Dummy` is not done — and saying so honestly earns marks.
- **Let Claude do the heavy lifting, then check it.** Ask it to scaffold the pipeline, then read every line and verify the numbers. Use **Context7** so the code uses current scikit-learn/XGBoost APIs.
- **Keep the app simple.** A clean predictor that loads beats a fancy one that crashes.
- **Stuck?** Paste the error to Claude and ask what it means — but make sure *you* understand the fix.

This is your chance to build something real, end to end. Make it something you would put in a portfolio. 